

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студент гр. 4343

Г.Д. Маркашов

Студент гр. 4343

Д.Е. Селезнев

Студент гр. 4343

Р.А. Пимахин

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

**ЗАДАНИЕ
НА УЧЕБНУЮ ПРАКТИКУ**

Студент: Г.Д. Маркашов

Студент: Д.Е. Селезнев

Студент: Р.А. Пимахин

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).
Необходимо найти минимальное остовное дерево для данного графа. С
использованием генетических алгоритмов

Ответственный за работу с данными, целостность доставки до пользователя
и алгоритмов - Г.Д. Маркашов

Ответственный за работу алгоритма - Д.Е. Селезнев

Ответственный за работу графической части ПО - Р.А. Пимахин

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студент

Г.Д. Маркашов

Студент

Д.Е. Селезнев

Студент

Р.А. Пимахин

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В данной работе рассматривается задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе с использованием генетического алгоритма. Целью проекта является разработка программного приложения с графическим интерфейсом пользователя, позволяющего задавать исходные данные, настраивать параметры алгоритма, выполнять поиск решения и визуализировать процесс оптимизации.

В отчёте описываются особенности постановки задачи, структура представления графа, принципы работы реализованного генетического алгоритма, используемые операции формирования популяции, отбора, скрещивания и мутации, а также функция качества решения. Рассматривается архитектура разработанного программного обеспечения, возможности взаимодействия пользователя с графическим интерфейсом и способы визуализации процесса поиска минимального остовного дерева.

Результатом работы является программное приложение на языке Python, позволяющее находить приближённое решение задачи МОД, отображать промежуточные этапы работы алгоритма, строить график изменения качества решений по поколениям и сравнивать различные варианты полученных решений.

SUMMARY

In this paper, we consider the problem of finding a minimum spanning tree (MOD) in a weighted undirected graph using a genetic algorithm. The goal of the project is to develop a software application with a graphical user interface that allows you to set source data, configure algorithm parameters, search for solutions, and visualize the optimization process.

The report describes the specifics of the problem statement, the structure of the graph representation, the principles of operation of the implemented genetic algorithm, the operations of population formation, selection, crossing and mutation used, as well as the quality function of the solution. The architecture of the developed software, the possibilities of user interaction with a graphical interface, and ways to visualize the process of searching for a minimum spanning tree are considered.

The result of the work is a Python software application that allows you to find an approximate solution to the MOD problem, display the intermediate stages of the algorithm, graph changes in the quality of solutions over generations, and compare different variants of the solutions obtained.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Предметная область	8
1.2 Задачи практики	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	10
2.1. Работа с данными. Проверка целостности и доставка данных от UI до алгоритма и обратно.	10
2.2. Генетический алгоритм для поиска МОД	15
2.3. Графический интерфейс пользователя (GUI) и визуализация	23
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	27
4 ОТЗЫВ РУКОВОДИТЕЛЯ	28
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	31

ВВЕДЕНИЕ

Предметной областью данной работы являются задачи оптимизации на графах, в частности задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе. Подобные задачи находят применение при проектировании сетей связи, транспортных систем, электрических сетей и других областях, где требуется построение наиболее эффективной структуры при минимальных затратах. Для решения сложных оптимизационных задач могут применяться эвристические методы, одним из которых являются генетические алгоритмы, основанные на моделировании процессов естественного отбора и эволюции.

Целью практики является разработка программного приложения для поиска минимального остовного дерева во взвешенном неориентированном графе с использованием генетического алгоритма. Разрабатываемая программа должна обеспечивать возможность задания исходных данных различными способами, настройки параметров алгоритма пользователем, визуального отображения процесса поиска решения и анализа эффективности работы алгоритма.

Для достижения поставленной цели необходимо решить следующие задачи: изучить особенности задачи поиска минимального остовного дерева и методы её решения; разработать представление графа и структуру генетического алгоритма; реализовать основные этапы работы алгоритма, включая формирование начальной популяции, оценку качества решений, отбор, скрещивание и мутацию; разработать графический интерфейс пользователя на языке Python; реализовать возможность ввода данных через интерфейс, загрузки из файла и генерации случайных графов; обеспечить пошаговую визуализацию процесса поиска решения, построение графика изменения функции качества и возможность получения итогового результата без просмотра всех промежуточных этапов.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью практики являются поиск минимального остовного дерева во взвешенном неориентированном графе с использованием генетических алгоритмов. Подобные задачи применяются при проектировании сетей связи, транспортных систем, электрических сетей и других структур, где требуется найти наиболее эффективное соединение объектов с минимальными затратами.

В рамках практики рассматривается применение генетических алгоритмов для решения задач. Разрабатываемое программное приложение позволяет исследовать процесс поиска решения, настраивать параметры алгоритма и визуализировать этапы его работы.

1.2 Задачи практики

В рамках прохождения практики были поставлены следующие задачи:

1. Изучить особенности задачи поиска минимального остовного дерева во взвешенном неориентированном графе и существующие методы её решения.
2. Изучить принципы работы генетических алгоритмов и возможности их применения для решения задач поиска оптимальных решений.
3. Изучить средства разработки графического интерфейса на языке Python, в частности библиотеку Tkinter, а также инструменты визуализации графов с использованием библиотеки NetworkX.
4. Разработать структуру представления графа и алгоритм поиска минимального остовного дерева на основе генетического алгоритма.
5. Реализовать основные этапы работы генетического алгоритма: создание начальной популяции, вычисление функции качества, отбор, скрещивание и мутацию.
6. Разработать графический интерфейс программы с использованием Tkinter, обеспечивающий ввод данных, загрузку графов из файла, генерацию случайных графов и настройку параметров алгоритма.

7. Реализовать визуализацию графа и процесса поиска решения с использованием NetworkX, включая отображение промежуточных результатов, текущих решений и изменения качества решений по поколениям.

8. Провести тестирование разработанного приложения и оценить корректность работы алгоритма на различных входных данных.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Работа с данными. Проверка целостности и доставка данных от UI до алгоритма и обратно.

Была проведена работа над pydantic схемами для валидации и хранения данных. В ходе проведения разработки ПО были созданы следующие классы:

`class GAStateStep(BaseModel)` для хранения информации о текущем шаге. Он имеет следующие поля:

- `generation` – номер поколения
- `population` – список хромосом(кодов Прюфера)
- `best_fitness` – лучшее значение качества в поколении
- `avg_fitness` – среднее значение качества в поколении
- `worst_fitness` – худшее значение качества в поколении
- `class GAHistory(BaseModel)` для хранения истории работы алгоритма (история шагов эволюции) со следующими полями:
 - `generations_count` – кол-во поколений
 - `best_fitness_history` – список лучших значений качества в поколениях
 - `avg_fitness_history` – список средних значений качества в поколениях
 - `worst_fitness_history` – список худших значений качества в поколениях
 - `steps` – список шагов эволюции (`GAStateStep`)
 - Класс имеет несколько методов:
 - `add_step(self, step: GAStateStep)` для добавления шага в историю
 - `get_step(self, index: int) -> GAStateStep` для получения шага по индексу
 - `get_last_step()` для получения последнего шага

- `class GraphData(BaseModel)` для хранения информации о графе. Имеет следующие поля:
- `num_vercites` – кол-во вершин в графе
- `matrix` – матрица смежности

Для корректности работы алгоритмов были установлено несколько валидаций данных и проверки на их соблюдения. Именно для этих целей был создан метод `validate_matrix` который срабатывает сразу после создание объекта и проверяет граф на:

1. Наличие петель и в случае их обнаружения выбрасывает исключение `ValueError`
2. Использование только ориентированного графа, в случае исключения выбрасывает исключение `ValueError`

Для удобства работы с алгоритмами и конвертации данных были реализованы следующие методы:

- `from_edges_list(n: int, edges: list[tuple[int, int, float]])` для создания объекта класса с использованием списка ребер
- `to_edges_list` обратная функция для извлечения списка ребер из графа.

Class `Graph` для управления данными графа и извлечению, установки и применения графа в алгоритме. Конструктор класса принимает кол-во вершин и создает объекта класса `GraphData` и хранит кол-во вершин. Для управления графом были реализованы следующие функции:

- `get_edges` – метод возврата списка ребер из графа
- `get_matrix` – метод возврата списка смежности графа
- `get_data` – метод возврата данных графа из объекта `GraphData`
- `_build_adj_list` – защищенный метод для формирования списка смежности

- `_validate_vertex` – метод для валидации вершины по допустимому диапазону значений
- `add_edge` – метод по добавлению ребра в граф
- `find_edge` – метод по поиску ребра в графе
- `has_edge` – метод по проверки существования ребра
- `get_edge_weight` – метод по получению веса ребра
- `get_copy_edges` – метод возвращаемый копия списка ребер
- `get_copy_neighbors` – метод по возвращению копии списка соседей(списка смежности)
- `get_degree` – метод возвращающий степень вершины
- `get_total_edges` – метод возвращающий кол-во ребер в графе
- `get_total_weight` – метод возвращающий вес всех ребер графа в сумме
- `get_edges_weight` – метод возвращающий вес определенных ребер, передаваемых списком кортежей ребер
- `to_dict` – метод миграции данных о графе в словарь
- `to_json` – метод миграции данных в json файл
- `is_connected` – метод проверки связности графа
- `is_tree` – статический метод класса для проверки списка ребер на образование дерева из этих ребер

`class PruferCode` один из важнейших классов в ПО. Класс призван для кодировании и декодировании графа в код Прюфера из ребер и наоборот из кода Прюфера в список ребер. Имеет следующие методы:

- `edges_to_code` – статический метод класса для кодирования списка ребер в код Прюфера
- `code_to_edges` – статический метод для декодирования кода Прюфера в список ребер

- `is_valid_prufer_code` – статический метод для проверки влидности кода Прюфера

`class FileIO`. Класс предназначен для работы с файлами, а именно для чтения графа из файла в формате списка ребер. Класс имеет следующий метод:

- `load` – статический метод считывающий данные из файла, их валидация и создания объекта класса `Graph` с использованием полученных ребер и последующим возврата объекта класса `Graph`

`class GraphGenerator`. Класс для генерации графа с использованием параметров. Класс имеет следующий метод:

- `gen_graph` – статический метод графа с параметрами: кол-во вершин, вероятность появления ребра между вершинами, минимальным и максимальным весам ребра

`class GraphManager`. Класс абстракции объединяющий несколько классов в один для удобного управления и комбинации использований других классов, а также для валидации приходящих данных. Метод конструктор объекты классов `FileIO` и `GraphGenerator`, а также поле `_graph` для хранения графа. Класс представляет собой менеджер управления графом. Имеет следующие методы:

- `get_graph` – возвращает объект метода `Graph` при его наличии
- `has_graph` – проверяет есть ли граф в менеджере
- `set_graph` – метод установки графа в менеджере
- `clear` – метод по удалению графа из менеджера
- `load_from_tuple` – метод по созданию и добавления графа в менеджер при входных данных списка ребер с весами
- `generate_random_graph` – метод по генерации графа с использованием другого класса

- `get_edges` – метод возврата списка ребер из графа
- `get_matrix` – метод возврата списка смежности графа
- `get_data` – метод возврата данных графа из объекта `GraphData`
- `add_edge` – метод по добавлению ребра в граф
- `find_edge` – метод по поиску ребра в графе
- `has_edge` – метод по проверки существования ребра
- `get_edge_weight` – метод по получению веса ребра
- `get_copy_edges` – метод возвращаемый копия списка ребер
- `get_copy_neighbors` – метод по возвращению копии списка соседей(списка смежности)
- `get_degree` – метод возвращающий степень вершины
- `get_total_edges` – метод возвращающий кол-во ребер в графе
- `get_total_weight` – метод возвращающий вес всех ребер графа в сумме
- `get_edges_weight` – метод возвращающий вес определенных ребер, передаваемых списком кортежей ребер
- `graph_to_dict` – метод миграции данных о графе в словарь
- `graph_to_json` – метод миграции данных в json файл
- `graph_is_connected` – метод проверки связности графа

`class HistoryManager`. Класс по хранению и управлению историей эволюции особей, шагов алгоритма. В конструкторе класс создает объекта класса `GAHistory`. Класс имеет следующие методы:

- `add_steps` – метод по добавлению шага историю
- `go_forward` – метод по переходу на следующий шаг в истории
- `go_back` – метод по переходу на шаг назад в истории
- `go_to` – метод по переходу на конкретный шаг в истории
- `go_to_end` – метод по переходу на последний шаг в истории
- `go_to_start` – метод по переходу в начало истории

- `get_current` – метод по извлечению данных из истории с текущего шага
- `get_step` – метод по получению шага по индексу
- `get_all` – метод получения копии всей истории
- `get_history_stats` – метод по получению статистики
- `get_last` – метод по получению информации из последнего шага
- `get_first` – метод по получению информации из первого шага
- `clear` – метод по очистки истории
- `is_empty` – метод по проверки пустоты истории

`class CrossoverType`. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

`class MutationType`. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

`class SelectionType`. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

2.2. Генетический алгоритм для поиска МОД

`class GeneticAlgorithmMST` – класс, управляющий процессом эволюции популяции решений для поиска минимального остовного дерева. Обеспечивает полный цикл генетического алгоритма, включая инициализацию популяции, оценку приспособленности особей, применение генетических операторов, сохранение истории для отката и сбор статистики для анализа сходимости.

Поля класса:

- `graph: Graph` – граф, для которого ищется минимальное остовное дерево.
- `population_size: int` – размер популяции (количество особей в каждом поколении).
- `mutation_rate: float` – вероятность мутации особи.
- `crossover_rate: float` – вероятность скрещивания родительских особей.
- `elitism_count: int` – количество лучших особей, переходящих в следующее поколение без изменений.
- `selection_type: str` – тип селекции ("tournament" для турнирной или "roulette" для рулеточной).
- `crossover_type: str` – тип скрещивания ("uniform" для равномерного, "one_point" для одноточечного или "two_point" для двухточечного).
- `mutation_type: str` – тип мутации ("swap" для мутации обменом или "random_reset" для мутации заменой).
- `tournament_size: int` – размер турнира для турнирной селекции.
- `operators: GeneticOperators` – экземпляр класса генетических операторов для выполнения селекции, скрещивания и мутации.
- `population: List[List[int]]` – текущая популяция особей (кодов Прюфера).
- `history: List[Dict]` – история состояний алгоритма для возможности отката к предыдущим поколениям.
- `generation: int` – номер текущего поколения.
- `best_fitness_history: List[float]` – список лучших значений `fitness` для каждого поколения.
- `avg_fitness_history: List[float]` – список средних значений `fitness` для каждого поколения.
- `worst_fitness_history: List[float]` – список худших значений `fitness` для каждого поколения.
- `max_possible_tree_weight: float` – кэшированное максимально возможное значение веса дерева, используемое для расчёта штрафов.

- `default_patience`: int – адаптивное значение терпения для проверки сходимости, вычисляемое как 30% от размера популяции, но не менее 30 и не более 150.

Методы класса:

- `__init__(num_vertices, mutation_rate=0.1, crossover_rate=0.8)` – инициализирует операторы с параметрами графа и вероятностями генетических операций, сохраняя все параметры в соответствующих полях класса для последующего использования.

- `__init__(graph, population_size=100, mutation_rate=0.1, crossover_rate=0.8, elitism_count=2, selection_type="tournament", crossover_type="uniform", mutation_type="swap", tournament_size=3)` – инициализирует алгоритм с заданными параметрами, проверяет связность графа, вычисляет максимально возможный вес дерева и адаптивное значение терпения, создаёт экземпляр операторов и инициализирует начальную популяцию.

- `_initialize_population()` – создаёт начальную популяцию случайных особей с помощью метода создания случайной особи из операторов и сохраняет начальное состояние в историю.

- `_calculate_fitness(individual)` – вычисляет функцию приспособленности для особи, используя двухуровневую систему оценки. Преобразует код Прюфера в рёбра, проверяет валидность каждого ребра через граф и возвращает суммарный вес для валидных деревьев или штрафное значение для невалидных, где штраф включает удвоенный максимальный вес дерева, тысячу за каждое невалидное ребро и суммарный вес валидных рёбер.

- `_evaluate_population()` – оценивает всю текущую популяцию, вызывая метод вычисления `fitness` для каждой особи и возвращая список пар особь-`fitness`.

- `_select_parent(evaluated_population)` – выполняет селекцию одного родителя из оценённой популяции в зависимости от выбранного типа

селекции, вызывая соответствующий метод операторов и возвращая копию выбранной особи.

- `_crossover(parent1, parent2)` – применяет оператор скрещивания к двум родительским особям в зависимости от выбранного типа, вызывая соответствующий метод операторов и возвращая пару потомков.

- `_mutate(individual)` – применяет оператор мутации к особи в зависимости от выбранного типа мутации, вызывая метод `'mutate_swap'` для типа "swap" или метод `'mutate'` для типа "random_reset" и возвращая мутированную особь.

- `_save_state()` – сохраняет текущее состояние алгоритма в историю, создавая словарь с номером поколения, копиями всех особей популяции и копиями списков статистики.

- `step()` – выполняет один шаг эволюции (одно поколение), включая оценку популяции, сбор статистики, применение элитизма и генерацию новых особей через селекцию, скрещивание и мутацию, замену популяции, инкремент счётчика поколений, сохранение состояния и возврат словаря со статистикой поколения.

- `has_converged(patience=None, min_improvement=0.001)` – проверяет достижение сходимости путём анализа последних поколений, используя адаптивное значение терпения при None, проверяя достаточность данных, извлекая последние значения лучшего fitness и вычисляя относительное улучшение для определения сходимости.

- `run(max_generations)` – запускает алгоритм на заданное количество поколений, выполняя цикл вызовов метода `step` и возвращая статистику последнего поколения. В текущей реализации проверка сходимости закомментирована, поэтому алгоритм всегда выполняет полное количество поколений.

- `get_best_solution()` – возвращает информацию о лучшем решении из текущей популяции, включая код Прюфера, список рёбер, вес дерева и номер поколения.

- `get_population_solutions(top_k=10)` – возвращает список из нескольких лучших решений текущей популяции с информацией о коде Прюфера, рёбрах и весе для каждого решения.

- `get_statistics()` – возвращает полную статистику работы алгоритма, включая текущий номер поколения и копии списков лучшего, среднего и худшего `fitness` по всем поколениям.

- `rollback(steps=1)` – выполняет откат состояния алгоритма на указанное количество шагов назад, проверяя возможность отката, вычисляя целевой индекс, восстанавливая состояние из истории и обрезаая более поздние состояния, возвращая `True` при успехе или `False` при невозможности отката.

- `reset()` – полностью сбрасывает алгоритм в начальное состояние, обнуляя счётчик поколений, очищая историю и статистику, генерируя новую начальную популяцию.

Модуль `operators.py`

Модуль содержит реализацию генетических операторов: селекции, скрещивания и мутации. Все операторы инкапсулированы в класс `GeneticOperators`, обеспечивающий модульность и возможность независимого тестирования.

`class GeneticOperators` – класс, инкапсулирующий все генетические операторы для работы алгоритма. Обеспечивает создание случайных особей, селекцию родителей, скрещивание и мутацию кодов Прюфера с учётом заданных вероятностей.

Поля класса:

- `num_vertices: int` – количество вершин в графе, используемое для генерации кодов Прюфера корректной длины.

- `mutation_rate: float` – вероятность выполнения мутации особи.

- `crossover_rate: float` – вероятность выполнения скрещивания родительских особей.

Методы класса:

- `__init__(num_vertices, mutation_rate=0.1, crossover_rate=0.8)` – инициализирует операторы с параметрами графа и вероятностями генетических операций, сохраняя все параметры в соответствующих полях класса для последующего использования.

- `create_random_individual()` – генерирует случайную особь в виде кода Прюфера длиной `num_vertices-2`, где каждый элемент представляет случайную вершину от 1 до `num_vertices`.

- `tournament_selection(population, tournament_size=3)` – реализует турнирную селекцию для выбора родителя. Использует `random.sample(population, min(tournament_size, len(population)))` для случайного выбора участников турнира, где количество ограничено минимумом из размера турнира и размера популяции для корректной работы на малых популяциях. Из выбранных участников находит победителя через `min(tournament, key=lambda x: x[1])`, где лямбда-функция извлекает второй элемент кортежа (`fitness`) как ключ сравнения, что выбирает особь с наименьшим `fitness`. Возвращает копию кода Прюфера победителя через `winner[0].copy()`. Турнирная селекция обеспечивает баланс между исследованием и эксплуатацией: больший размер турнира усиливает селекционное давление в пользу лучших решений.

- `roulette_selection(population)` – реализует рулеточную селекцию с инверсией `fitness` для задачи минимизации. Находит максимальный `fitness` в популяции через `max(fitness)`. Далее создаётся список инвертированных `fitness` через списковое включение, где каждое значение вычисляется как максимум минус `fitness` плюс единица, что преобразует задачу минимизации в максимизацию и обеспечивает положительность всех значений. Вычисляет сумму инвертированных `fitness`. Генерирует случайное вещественное число от 0 до суммы через `random.uniform(0, total_fitness)`, представляющее позицию на рулетке. Инициализирует накопитель и в цикле проходит по индексам и значениям инвертированных `fitness`. На каждой итерации к накопителю добавляется текущее значение `fitness`, и проверяется условие: если накопитель

превысил позицию на рулетке, возвращается копия особи по текущему индексу. Если цикл завершается без возврата из-за ошибок округления, возвращается копия последней особи как запасной вариант. Рулеточная селекция даёт всем особям шанс на выбор пропорционально их приспособленности.

- `uniform_crossover(parent1, parent2)` – реализует равномерное скрещивание родителей. Проверяется условие: если сгенерированное случайное число больше вероятности скрещивания, метод возвращает копии обоих родителей без изменений. Проверяет длину первого родителя через `if len(parent1) == 0` и возвращает пару пустых списков `[], []` для граничного случая. Создаёт два пустых списка для потомков. Далее запускается цикл по парам генов родителей через. На каждой итерации генерирует случайное число и проверяет условие: если условие выполнено, в первого потомка добавляется ген от первого родителя, а во второго потомка ген от второго родителя, иначе гены добавляются в обратном порядке. После обработки всех позиций возвращает пару сформированных потомков. Равномерное скрещивание обеспечивает равномерное смешивание генетического материала родителей на уровне отдельных генов.

- `one_point_crossover(parent1, parent2)` – реализует одноточечное скрещивание с одной точкой разреза. Проверяет вероятность скрещивания и возвращает копии родителей при отказе от скрещивания. Далее если длина родителей меньше или равна одному, возвращаются их копии, так как невозможно выбрать точку разреза для коротких кодов. Выбирает случайную точку разреза через, где точка выбирается в диапазоне от 1 до длины минус 1 включительно. После создаются потомки путём конкатенации срезов, получается начало до точки от первого родителя и хвост от точки от второго родителя получает начало от второго родителя и хвост от первого родителя. Возвращает пару созданных потомков. Одноточечное скрещивание сохраняет большие блоки генов, что может быть полезно, если структура кодов Прюфера имеет локальные зависимости.

- `two_point_crossover(parent1, parent2)` – реализует двухточечное скрещивание с двумя точками разреза. Проверяет условие выполнения скрещивания и возвращает копии родителей при отказе. Для родителей длиной 2 или меньше через делегирует выполнение методу `one_point_crossover`, так как невозможно выбрать две различные внутренние точки разреза. Выбирает первую точку разреза в диапазоне от 1 до длины минус 2, затем выбирает вторую точку в диапазоне от первой точки плюс один до длины минус один, что гарантирует упорядоченность точек и наличие среднего сегмента. Создаёт потомков путём конкатенации трёх срезов: `child1 = parent1[:point1] + parent2[point1:point2] + parent1[point2:]` и `child2` получает соответствующие части в обратном порядке. Возвращает пару сформированных потомков. Двухточечное скрещивание позволяет обмениваться внутренними сегментами кодов Прюфера, сохраняя граничные области от обоих родителей.

- `mutate(individual)` – реализует мутацию путём замены случайных генов на новые значения. Проверяет длину особи и возвращает копию если длина равна нулю. Далее создается копия особи и запускает цикл по всем позициям в мутированной копии через `for`. На каждой позиции генерирует случайное число и проверяет условие `if random.random() < self.mutation_rate`: если условие выполнено, ген на текущей позиции заменяется новым случайным значением, где новое значение выбирается от 1 до количества вершин включительно. После прохода по всем позициям возвращает мутированную копию, которая может содержать от нуля до всех изменённых генов в зависимости от вероятности мутации и случайности. Мутация заменой вводит новые гены в популяцию, что увеличивает разнообразие и помогает избегать локальных оптимумов.

- `mutate_swap(individual)` – реализует мутацию обменом двух случайных генов. Генерирует случайное число и проверяет условие: если число больше вероятности мутации или длина особи меньше двух, возвращает копию особи без изменений. Потом создаётся копия особи и выбирается два различных индекса через `i, j = random.sample(range(len(mutated)), 2)`, где `random.sample`

гарантирует выбор двух разных позиций из диапазона длины особи. Обменивает гены на выбранных позициях через одновременное присваивание. Возвращает мутированную особь `mutated` с обменёнными генами. Мутация обменом сохраняет набор генов, присутствующих в коде Прюфера, изменяя только их порядок, что может значительно изменить структуру кодируемого дерева.

Таким образом Класс `GeneticAlgorithmMST` использует экземпляр класса `GeneticOperators` для выполнения всех генетических операций в процессе эволюции. Основной алгоритм управляет высокоуровневой логикой эволюционного процесса, включая оценку популяции с использованием двухуровневой функции приспособленности, формирование новых поколений через элитизм и генерацию потомков, сохранение истории для возможности отката к предыдущим состояниям и сбор статистики для анализа сходимости. Генетические операторы инкапсулируют низкоуровневые операции над кодами Прюфера, обеспечивая модульность и возможность независимого тестирования каждого оператора.

Использование кодов Прюфера как представления решений гарантирует, что каждая особь кодирует корректное остовное дерево без циклов, что устраняет необходимость проверок валидности на структурном уровне. Двухуровневая функция приспособленности с системой штрафов направляет эволюцию от невалидных деревьев, содержащих рёбра отсутствующие в графе, через постепенное исправление ошибок к валидным деревьям, и далее к минимизации суммарного веса среди валидных решений. Такой подход обеспечивает работоспособность алгоритма даже при случайной инициализации популяции с большим количеством невалидных особей, позволяя алгоритму постепенно улучшать решения через механизмы селекции, скрещивания и мутации.

2.3. Графический интерфейс пользователя (GUI) и визуализация

Графический интерфейс пользователя (GUI) и визуализация

Графический интерфейс разработан с использованием стандартной библиотеки tkinter, а также библиотек matplotlib и network для динамической отрисовки графов.

В ходе разработки UI-модуля были созданы следующие классы:

```
class GraphDrawer
```

Класс отвечает за отрисовку структуры графа и выделение найденного минимального остовного дерева (МОД) на поле tkinter с помощью интеграции с matplotlib и networkx.

- Конструктор инициализирует фигуру Matplotlib, настраивает белый фон и интегрирует виджет графика в tk.Canvas родительского окна через FigureCanvasTkAgg.

- Методы класса:

- draw(self, vertices_count, edges_list, mst_edges=None, reset_layout=False) - основной метод отрисовки. Формирует граф средствами networkx, рассчитывает стабильное расположение вершин по алгоритму spring_layout. Отрисовывает ребра графа, вершины, их метки и веса. При передаче списка mst_edges подсвечивает ребра, входящие в текущее лучшее остовное дерево, красным цветом (crimson) повышенной толщины.

- clear(self) - полностью очищает координатные оси холста, сбрасывает кэш позиций вершин и обновляет полотно.

```
class GA_GUI
```

Центральный класс главного окна приложения. Он реализует панель настройки параметров генетического алгоритма, логирование, вывод статистики и управление исходными данными графа.

- Конструктор инициализирует главное окно, компоует интерфейс, настраивает текстовые поля для ввода параметров (размер популяции, поколения, вероятности операторов, элитизм) и связывает события элементов управления.

- Методы класса:

- `load_in_ga(self)` -> bool - проверяет, загружен ли граф, и инициализирует объект генетического алгоритма GeneticAlgorithmMST параметрами, считанными из полей ввода UI.
- `update_selection(self, event=None)` - динамически управляет доступностью поля «Размер турнира» в зависимости от выбранного метода селекции (активно только для турнирного отбора).
- `load_graph(self)` - вызывает стандартный диалог открытия файла, считывает граф через GraphManager и отображает его на холсте.
- `generate_graph(self)` - открывает модальное диалоговое окно для ввода параметров генерации (число вершин, плотность ребер, диапазон весов) случайного связного графа.
- `manual_input_graph(self)` - открывает окно для ручного текстового ввода списка ребер в формате `u v w`, осуществляет построчное считывание данных, валидацию формата и передачу ребер в менеджер.
- `reset(self)` - останавливает выполнение, сбрасывает текущие графики, очищает текстовые метки статистики и лог.
- `show_stats(self, ga)` - принимает объект алгоритма и обновляет текстовые метки лучшего, среднего и худшего значений фитнес-функции в текущем поколении на основе актуальной истории.
- `run_algorithm(self)` - запускает главный алгоритм, инициализирует эволюционный движок, делает тестовый запуск и открывает отдельное окно пошаговой визуализации EvolutionWindow.

`class EvolutionWindow`

Класс дополнительного окна, предназначенного для интерактивного контроля, автоматического проигрывания и пошагового анализа процесса эволюции хромосом.

- Конструктор принимает родительское окно, ссылку на объект GA, максимальное число поколений и функцию обратного вызова для обновления статистики. Создает холст для отрисовки графа и панель управления шагами.
- Методы класса:

- `update_plot(self)` - запрашивает у алгоритма хромосому лучшего решения в текущем поколении (набор ребер и их суммарный вес) и отправляет эти данные в `GraphDrawer` для обновления визуализации на холсте, параллельно обновляя информационные метки поколения.
- `next_step(self)` - выполняет ровно одну итерацию (поколение) генетического алгоритма с помощью метода `ga.step()` и перерисовывает граф.
- `prev_step(self)` - производит откат состояния эволюции на одно поколение назад (`ga.rollback()`).
- `skip_to_end(self)` - прерывает автоматическое проигрывание и мгновенно доводит алгоритм до финального поколения или критерия сходимости, отображая итоговый результат.
- `toggle_play(self)` - переключает режим автопроигрывания (Старт / Пауза) эволюции.
- `play_loop(self)` - цикл автоматического выполнения шагов алгоритма, использующий метод `window.after(500, ...)` для обеспечения задержки в 500 мс между поколениями.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

При выполнении практической работы использовались следующие технологии и программные средства:

Python 3 основной язык программирования, на котором реализованы генетический алгоритм и программное приложение.

Tkinter стандартная библиотека Python для создания графического интерфейса пользователя. Использовалась для разработки окон приложения, организации ввода исходных данных, настройки параметров генетического алгоритма и отображения результатов работы программы.

NetworkX библиотека для работы с графами. Применялась для создания, визуализации взвешенного неориентированного графа.

Rydantic библиотека для работы с данными, удобному обращению к ним и валидации их.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы была разработана полнофункциональная программная система для решения задачи поиска минимального остовного дерева взвешенного неориентированного графа с использованием генетического алгоритма.

Ядро системы представлено модулями генетического алгоритма и операторов, реализующими полный цикл эволюционного процесса с использованием кодов Прюфера в качестве представления решений. Применение кодов Прюфера обеспечило корректность всех генерируемых решений на структурном уровне, исключив необходимость проверок на ацикличность и связность деревьев. Двухуровневая функция приспособленности с адаптивной системой штрафов обеспечивает постепенную эволюцию от невалидных решений к оптимальным. Алгоритм поддерживает гибкую настройку параметров, включая выбор типов селекции, скрещивания и мутации, механизм элитизма и адаптивный критерий сходимости. Система сохранения истории обеспечивает возможность отката к любому предыдущему поколению.

Слой работы с данными построен на основе pydantic-схем, обеспечивающих строгую валидацию и целостность данных. Класс `'Graph'` предоставляет полный набор методов для работы с графами, класс `'PruferCode'` реализует алгоритмы кодирования и декодирования, классы `'GraphGenerator'` и `'FileIO'` обеспечивают генерацию тестовых данных и загрузку из файлов. Класс `'GraphManager'` объединяет функциональность в единый интерфейс, а `'HistoryManager'` предоставляет навигацию по истории эволюции.

Графический интерфейс, разработанный с использованием `tkinter`, `matplotlib` и `networkx`, предоставляет интуитивный способ взаимодействия с

системой. Класс `'GA_GUI'` реализует главное окно с панелью настройки параметров и отображением статистики. Класс `'GraphDrawer'` обеспечивает визуализацию графов с выделением рёбер минимального остовного дерева. Класс `'EvolutionWindow'` предоставляет интерактивный интерфейс для пошагового анализа процесса эволюции с возможностью ручного управления и автоматического проигрывания.

Разработанная система успешно прошла тестирование на различных графах, демонстрируя способность находить минимальные остовные деревья для связных взвешенных графов различной сложности. Модульная архитектура обеспечивает лёгкость расширения функциональности и добавления новых операторов. Использование современных практик разработки, включая строгую типизацию, валидацию данных и разделение ответственности между классами, гарантирует поддерживаемость системы.

Полученные результаты подтверждают эффективность применения генетических алгоритмов для решения задачи поиска минимального остовного дерева. Система может использоваться в образовательных целях для изучения эволюционных алгоритмов и в качестве основы для решения практических задач оптимизации сетевых структур. Реализованный функционал визуализации делает систему особенно ценной для исследовательских целей, позволяя детально изучать динамику генетического алгоритма и влияние различных параметров на качество и скорость сходимости к оптимальному решению.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.
3. Вирсански Э. В52 Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. – М.:
4. ДМК Пресс, 2020. – 286 с.: ил. Панченко, Т. В. Генетические алгоритмы [Текст] : учебно-методическое пособие / под ред. Ю. Ю. Тарасевича. — Астрахань : Издательский дом «Астраханский университет», 2007. — 87 [3] с.